Faculty of Engineering

Semester: Spring 2026 - Academic Year: 2025-2026

Module Title: Digital Signal Processing - Module Code: ECE465

Instructor Name: Dr. Nariman

TAs Name(s): Eng. Ahmed Hatem

Project Title: Animation (Image Processing)

Student ID: 231147 **Student Name: Omar Mohamed**

Table of Contents

1. Introduction..... 3
2. Problem Definition..... 3
3. Objectives..... 4
4. Methodology..... 4
5. Flowchart..... 6
6. Source Code..... 7
7. Conclusion..... 9
8. References..... 9

1. Introduction

Visual effects are essential for making digital images and videos feel realistic and professional. One classic example is the cinematic "focus pull," where the camera slowly transitions an image from heavily blurred to completely sharp. This technique naturally guides the viewer's eye and adds a lot of production value. For this project, we built a MATLAB simulation to replicate this exact effect using fundamental image processing techniques.

The process starts when the user selects an input photo. From there, the script generates a sequence of frames that behaves just like a physical camera lens searching for focus.

To make this happen, we relied primarily on a Gaussian blur filter. By controlling the 'sigma' parameter and gradually decreasing it over multiple frames, the script smoothly transitions the image from a soft blur to perfect sharpness.

However, we noticed that just applying a basic blur didn't look completely natural on its own. To make it feel like actual camera footage, we added a slight color channel shift to mimic chromatic aberration (lens distortion), and we introduced some random noise to simulate the imperfections usually caused by digital camera sensors. We programmed these extra effects to be intense when the image is heavily out of focus and slowly fade out as the picture clears up.

Under the hood, the entire sequence is processed frame by frame using convolution operations. Once the animation finishes displaying, the final sharp image locks in with a “Focus Locked” message on the screen.

Ultimately, this project was a great way to take theoretical concepts—like convolution, Gaussian filtering, and pixel manipulation—and actually combine them to build a realistic, cinematic visual effect.

2. Problem Definition

High-end video production uses expensive lenses and hardware to get those smooth, dynamic focus transitions. But if you are just working with standard digital images, it is surprisingly hard to replicate that cinematic feel computationally. Most basic image processing outputs just sit there looking entirely static or artificially sharp.

The main challenge we tackled in this project was finding a way to recreate a genuine "focus pull" entirely through code, without touching a physical camera. We needed to figure out how to take one static photo and programmatically generate a sequence of frames that shifts from blurry to sharp in a believable way.

The catch is that just throwing a basic blur over an image doesn't actually look real. A physical camera lens has natural quirks—like color shifting at the edges and a bit of grain or sensor noise when it's out of focus. If you ignore those physical characteristics, the output just looks like a cheap digital filter rather than a real lens adjustment.

To bridge that gap, we couldn't just stop at a simple Gaussian blur. We had to build a system that processes the image frame-by-frame, intentionally misaligns the color channels slightly to mimic lens distortion, and adds controlled mathematical noise. By combining all these elements, we were able to trick the eye into seeing a smooth, physical cinematic animation.

3. Objectives

Our primary goal for this project was to take a standard static image and turn it into a realistic cinematic focus pull using MATLAB. We wanted to build a script that programmatically shifts the image from heavily blurred to perfectly sharp, mimicking a real camera lens.

To get there, we broke the project down into a few specific technical milestones:

- Write a convolution-based Gaussian blur implementation to accurately control how sharp or soft the image appears.
- Create a smooth, natural-looking transition by dynamically changing the blur intensity (the sigma value) across a generated sequence of frames.
- Replicate physical lens distortion, specifically chromatic aberration, by slightly misaligning the color channels.
- Introduce a layer of controlled, randomized noise to mimic the grain you usually get from digital camera sensors when they hunt for focus.
- Set up a loop to process and render the output frame-by-frame so it actually plays like a video animation.
- Normalize all the pixel values throughout the mathematical operations to prevent any color clipping or quality drops.
- Show how core concepts—like spatial filtering, 2D convolution, and direct pixel manipulation—can be used together to build something highly visual and practical.

By hitting all these targets, we were able to take the digital signal processing theory we learned in class and turn it into a working, visually convincing camera simulation.

4. Methodology

To build this simulation, we set up a step-by-step image processing pipeline in MATLAB. We start by popping up a file selection interface so the user can grab any photo they want. Once loaded, the script pre-processes the image—resizing it to a manageable width so the convolution doesn't freeze the system, converting it to a three-channel RGB image

(dropping any transparency channel), and normalizing the pixel values to the range 0 to 1 so the math works out cleanly later on.

To create the actual video effect, we generate a loop of frames. Instead of just stepping down the blur linearly (which tends to look robotic and unnatural), we used a cosine function to control the blur parameter, sigma. This makes the focus pull start slowly, speed up in the middle, and then ease smoothly into the final sharp image, exactly like a camera operator twisting a physical lens ring.

Inside the loop, the code builds a custom Gaussian blur kernel for every single frame based on that changing sigma value, truncated at three standard deviations. Because a 2D Gaussian is separable, the kernel is applied as two 1D passes (down the columns, then along the rows) with conv2, which gives the same result as a full 2D kernel at a fraction of the computation. We apply this filter across all three color channels, blending each pixel with its neighbors to create the soft focus. Before filtering, the image is padded by replicating its border pixels so the edges do not darken.

However, a mathematically perfect blur doesn't look like actual film. To fix this, we added some physical lens characteristics. First, we offset the red and blue color channels horizontally by a few pixels based on the blur intensity, giving us a realistic chromatic aberration effect (the edge columns are replicated rather than wrapped around, so no pixels reappear on the opposite side). Next, we injected a randomized noise matrix into the frame to simulate the grainy look of a camera sensor struggling in low light. As the focus locks in and the sigma drops, both the color shifting and the noise scale back naturally.

Because all this math can push pixel values out of bounds, we clip everything back between 0 and 1 before rendering. We then force MATLAB to display the frames sequentially, with a short pause between them, generating a smooth animation.

When the loop wraps up, the script displays the crisp, original image with a "Focus Locked" overlay. By bringing all these pieces together—convolution, dynamic Gaussian kernels, noise matrices, and channel shifting—we were able to turn standard static filtering into a convincing cinematic shot.

5. Flowchart

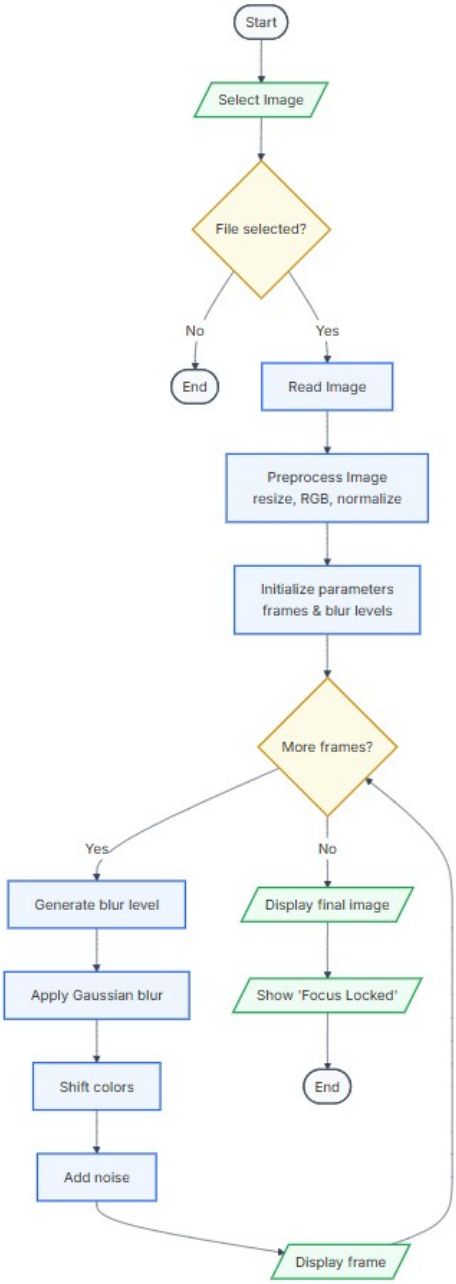


Figure 1: Flowchart of the cinematic focus pull image processing system.

6. Source Code

```
[filename, pathname] = uigetfile({'*.jpg;*.jpeg;*.png', 'Image Files'}, 'Select your photo');
if isequal(filename, 0)
    disp('No file selected. Exiting...');
    return;
end

img = imread(fullfile(pathname, filename));
if size(img, 3) == 4, img = img(:, :, 1:3); end          % drop alpha channel (PNG)
if size(img, 3) == 1, img = repmat(img, [1, 1, 3]); end  % grayscale -> RGB
if size(img, 2) > 600, img = imresize(img, [NaN, 600]); end
img = im2double(img);   % scale to [0,1] (works for uint8 and uint16 images)

figure('Name', 'Cinematic Focus Pull', 'Position', [100, 100, 600, 500], 'Color', 'k');

frames = 30;
t = linspace(0, pi, frames);
sigmas = 14 * ((cos(t) + 1) / 2) + 0.1;

for i = 1:frames
    sigma = sigmas(i);

    % Gaussian kernel truncated at 3*sigma (separable: 2D kernel = g * g')
    halfSize = ceil(3 * sigma);
    x = -halfSize:halfSize;
    g = exp(-x.^2 / (2 * sigma^2));
    g = g / sum(g);

    % Replicate border pixels so the image edges do not darken
    padded = padarray(img, [halfSize, halfSize], 'replicate');
    processedImg = zeros(size(img));
    for c = 1:3
        processedImg(:, :, c) = conv2(g, g, padded(:, :, c), 'valid');
    end

    % Chromatic aberration: red shifts left, blue shifts right (edges replicated)
    shiftAmount = round(sigma * 0.7);
    if shiftAmount > 0
        R = processedImg(:, :, 1);
        B = processedImg(:, :, 3);
        processedImg(:, :, 1) = [R(:, shiftAmount+1:end), repmat(R(:, end), 1, shiftAmount)];
        processedImg(:, :, 3) = [repmat(B(:, 1), 1, shiftAmount), B(:, 1:end-shiftAmount)];
    end

    noiseLevel = (sigma / 15) * 0.06;
    noiseMatrix = noiseLevel * randn(size(processedImg));
    finalFrame = processedImg + noiseMatrix;

    finalFrame = max(0, min(1, finalFrame));

    imshow(finalFrame);
    title('Searching for focus...', 'FontSize', 14, 'Color', [0.8 0.8 0.8]);
    drawnow;
    pause(0.05);
end

imshow(img);
title('Focus Locked.', 'FontSize', 16, 'Color', [0 1 0]);
disp('Cinematic animation complete!');
```

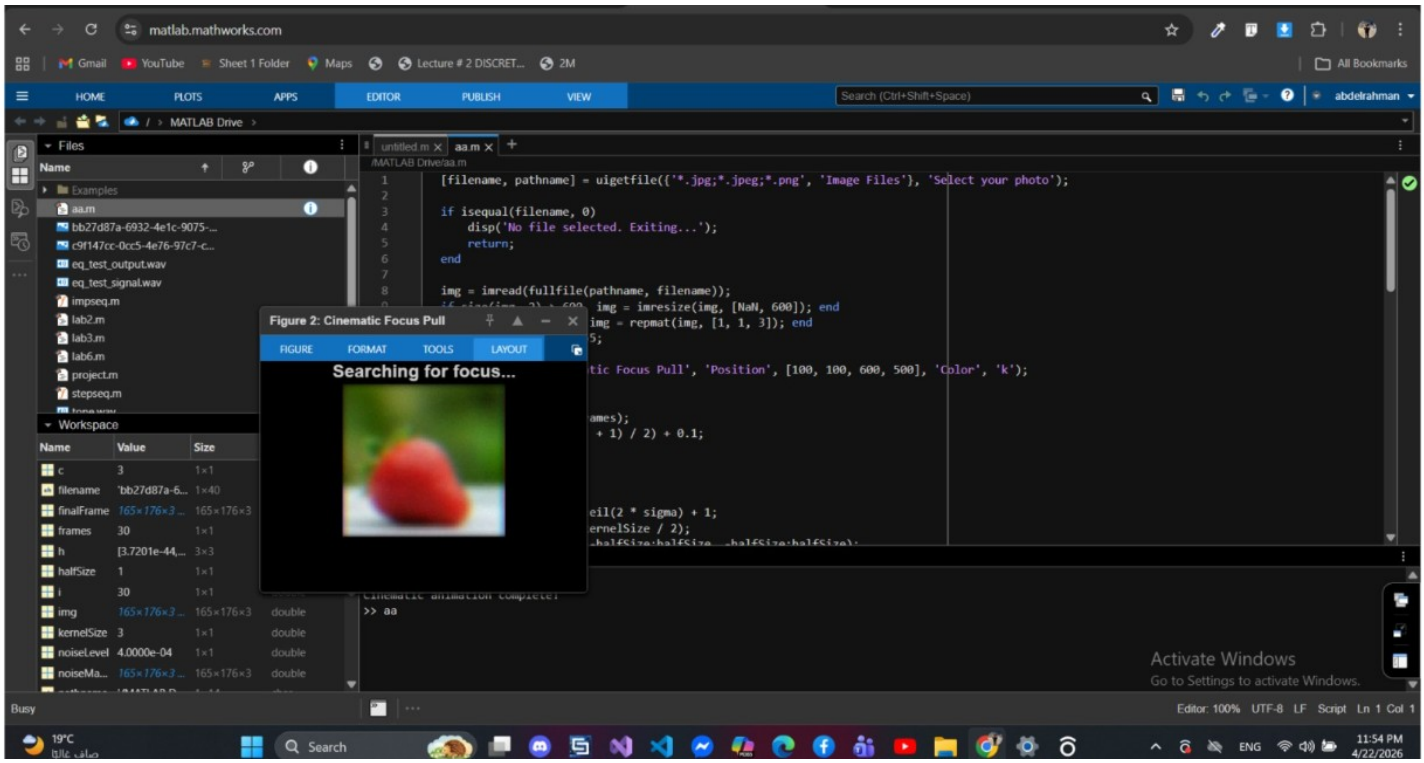


Figure 2: MATLAB output - a blurred, noisy frame during the focus pull ("Searching for focus...").

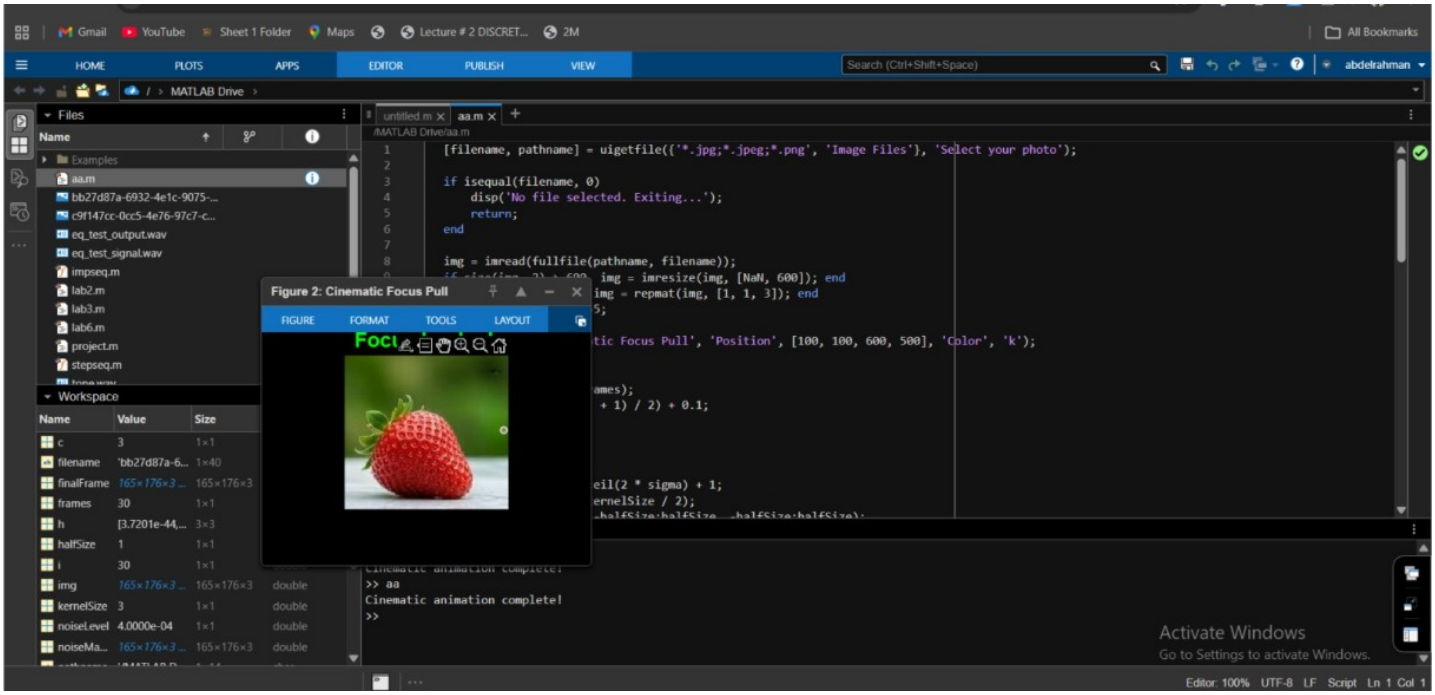


Figure 3: MATLAB output - final sharp frame with the "Focus Locked." title.

7. Conclusion

Ultimately, this project proved that we could successfully build a cinematic focus pull effect from scratch using fundamental MATLAB programming. Rather than relying on pre-packaged video editing software, we took a single static image and mathematically transformed it into a dynamic animation that actually behaves like physical camera hardware.

Relying on a dynamic sigma value to scale our Gaussian kernel gave us incredibly smooth control over the focus transition. However, adding the physical lens quirks—specifically the color channel shifting and the randomized sensor noise—was really what pushed the result from a basic digital blur to a realistic piece of footage.

This project was a highly practical way to take the core DSP concepts we covered in class—like spatial filtering, 2D convolution, and direct pixel manipulation—and apply them to something creative. It shows that with the right mathematical modeling, you can replicate high-end cinematic visuals using nothing but code.

8. References

1. Gonzalez, R. C., & Woods, R. E. (2018). *Digital Image Processing* (4th ed.). Pearson.
2. Oppenheim, A. V., & Schafer, R. W. (2010). *Discrete-Time Signal Processing* (3rd ed.). Pearson.
3. Szeliski, R. (2022). *Computer Vision: Algorithms and Applications* (2nd ed.). Springer.
4. MathWorks. *conv2 - 2-D convolution*, MATLAB Documentation. https://www.mathworks.com/help/matlab/ref/conv2.html
5. MathWorks. *padarray - Pad array*, Image Processing Toolbox Documentation. https://www.mathworks.com/help/images/ref/padarray.html